



HOTEL RESERVATION & BILLING MANAGEMENT

Team Members :

Rahma Ahmed Hussien | Rowan Harpy | Rawan Mohamed | Gehan Sultan | Rehab Moawad |
zahraa Moustafa

SYSTEM

Full Data Pipeline Project Report

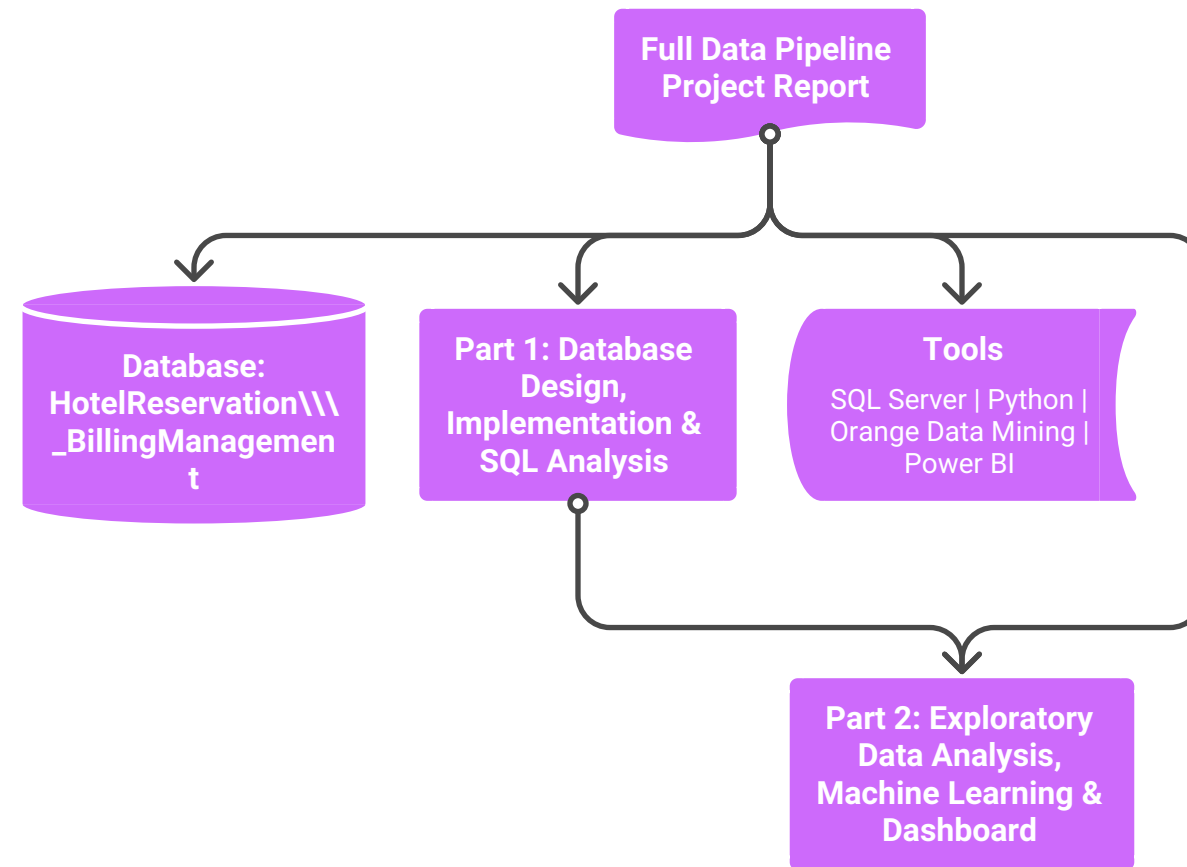
Part 1: Database Design, Implementation & SQL Analysis

Part 2: Exploratory Data Analysis, Machine Learning & Dashboard

Database: HotelReservation_BillingManagement

Tools: SQL Server | Python | Orange Data Mining | Power BI

Hotel Reservation & Billing Management System Data Pipeline



1. Project Overview

1.1 Introduction

This project presents a comprehensive Hotel Reservation and Billing Management System designed to digitize and streamline all core operations of a modern hotel. In today's competitive hospitality industry, hotels generate enormous volumes of data — from guest check-ins and room assignments to service usage and billing. Without a structured database system, managing these operations manually leads to errors, revenue leakage, and poor guest experience.

Our project addresses this challenge by designing a relational database, implementing it in Microsoft SQL Server, and extracting strategic business insights through analytical queries, Python-based EDA, machine learning modeling in Orange, and an interactive Power BI dashboard.

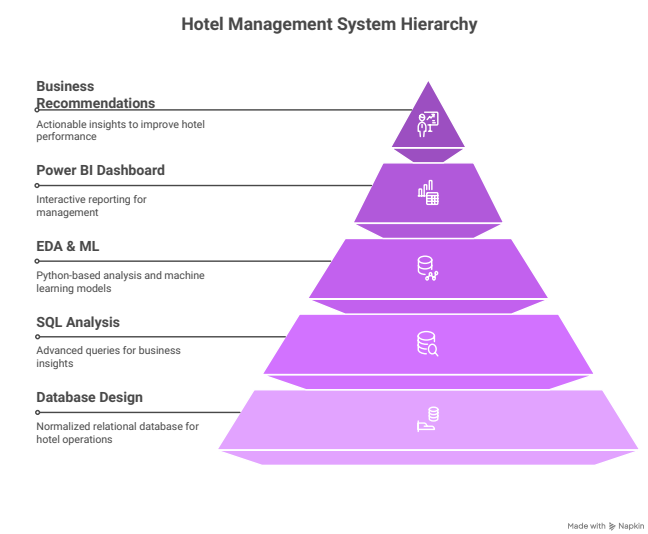
1.2 Objectives

- Design a fully normalized relational database covering all hotel operations.
- Implement the database in SQL Server with proper constraints, keys, and relationships.
- Write advanced SQL queries (views, CTEs, window functions) to answer critical business questions.
- Perform Exploratory Data Analysis (EDA) using Python (Jupyter Notebook) to uncover patterns.

- Build and evaluate machine learning models in Orange Data Mining to predict booking behavior.
- Create an interactive Power BI dashboard for management reporting and KPI tracking.
- Derive actionable business recommendations to improve occupancy, revenue, and guest satisfaction.

1.3 Scope of the System

The system covers the complete hotel lifecycle: from guest registration and room booking through check-in, housekeeping, service usage, invoicing, and payment processing. The database supports two physical buildings, multiple room types (Single, Double, Suite), a loyalty rewards program, staff performance tracking, and review management.



2. Entity Relationship Diagram (ERD)

2.1 Why We Designed This ERD

Before writing a single line of SQL, we invested significant effort in designing the ERD. The reason is straightforward: a poor database design causes data redundancy, update anomalies, and query inefficiency that are almost impossible to fix later. We applied Third Normal Form (3NF) principles to eliminate redundancy and ensure every attribute depends on its primary key alone.

2.2 Entities and Their Roles

The ERD contains 12 core entities, each representing a distinct real-world concept in the hotel:

Field	Description
Guests	Stores all guest profiles including name, contact, city, and national ID. Every person who makes a reservation must exist here first.
Reservation	The central entity. Records every booking with check-in/out dates, status (confirmed, checked_in, checked_out, cancelled, no_show), room assignment, and guest reference.
Rooms	Physical rooms in the hotel. Each room has a number, availability status, type reference, and building reference.
RoomsType	Master table for room categories: Single (500/night), Double (900/night), Suite (1,500/night). Separates pricing from physical rooms.
Buildings	Two hotel buildings [B1, B2] with name, address, and floor count. Rooms belong to buildings.
Invoice	Generated per reservation. Tracks total room charge, service charges, discounts, and net payable amount.
Payments	Records every payment transaction against an invoice. Supports Cash, Card, Online, and Wallet methods.
Services	Master list of hotel services (Spa, Laundry, Breakfast, Dinner, etc.) with per-unit pricing.
ServiceUsage	Junction table recording which services a guest used during which reservation, with quantity and total charge.
Employee	All hotel staff with job roles (Receptionist, Billing Manager, Admin) and personal details.
HousekeepingLogs	Tracks every room cleaning session: who cleaned it, when, start/end time, and cleaning date.
LoyaltyProgram	Tracks guest loyalty points and membership level (Bronze, Silver, Gold). Links to Guests.

2.3 Key Relationships

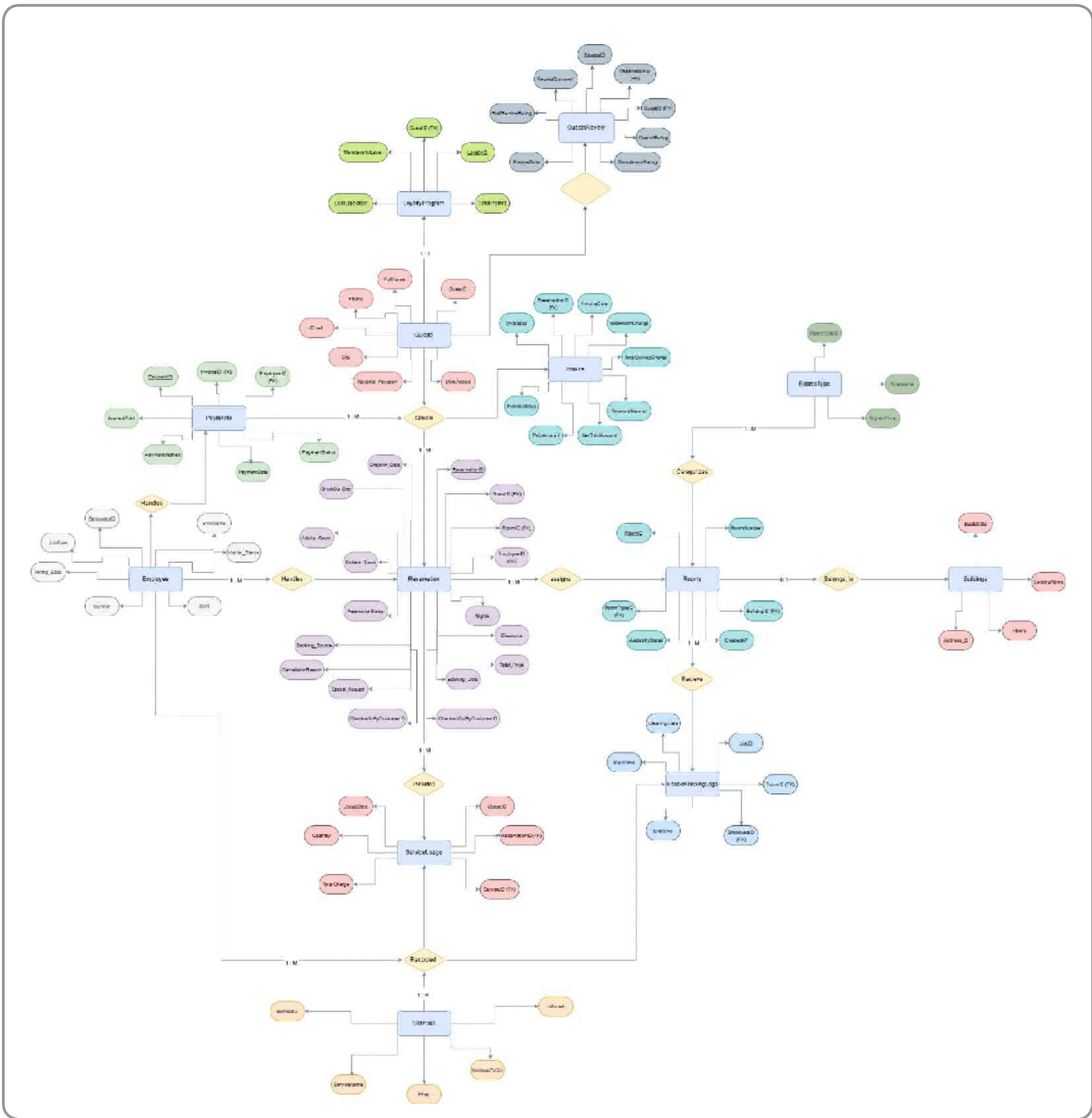
The relationships between entities encode the business rules of the hotel:

- Guest [1] → Reservation [M]: One guest can make multiple reservations over time, but each reservation belongs to exactly one guest.
- Reservation [1] → Invoice [1]: Each completed reservation generates exactly one invoice.
- Invoice [1] → Payments [M]: An invoice can be settled through multiple payment transactions.
- Reservation [M] → Rooms [1]: Each reservation is assigned one room; a room can be reserved many times [sequentially].
- Rooms [M] → RoomsType [1]: Many rooms share the same type. Type drives nightly pricing.
- Rooms [N] → Buildings [1]: Each room belongs to one building; buildings contain many rooms.
- Reservation [1] → ServiceUsage [M]: A reservation can include multiple service usages.
- ServiceUsage [M] → Services [1]: Each usage references one service from the master list.
- Employee [1] → Reservation [M]: Each reservation is handled by a staff member. Check-in and check-out can be recorded by different employees.
- Rooms [1] → HousekeepingLogs [M]: Every room accumulates multiple cleaning sessions.
- Guest [1] → LoyaltyProgram [1]: Each guest has exactly one loyalty record.

- Guest [1] → GuestsReview [M]: Guests may submit multiple reviews across different stays.

2.4 Design Assumptions

- A room can only be assigned to ONE active reservation at any point in time [enforced by application logic and reservation_status].
- Nightly pricing is captured at booking time in the Reservation table [NightlyRateAtBooking] in addition to the RoomsType reference, to handle future price changes without altering historical data.
- Services can be added to a reservation even after check-in, as long as the reservation is still active.
- Payments can be partial — the invoice tracks PaidAmount separately from NetTotalAmount, enabling outstanding balance reporting.
- Employees are assigned job roles that determine what operations they can perform [Receptionists handle check-ins, Billing Managers process payments].
- The LoyaltyProgram is automatically created when a guest profile is first added, starting at Bronze level with zero points.



3. Database Implementation

3.1 Why SQL Server?

We chose Microsoft SQL Server for this project because it provides robust support for relational constraints (FK, PK, CHECK), advanced query features (CTEs, window functions, views), and seamless integration with Power BI for dashboard development. SQL Server's transaction management ensures data integrity across multi-table operations like invoice creation and payment processing.

3.2 Tables and Their Structure

All tables were created with explicit Primary Keys (PK), Foreign Keys (FK), NOT NULL constraints, and CHECK constraints where appropriate. Below is a summary of the implementation logic for each table:

3.2.1 Core Tables

- Guests: PK = GuestID (VARCHAR), UNIQUE constraint on NationalID_Passport to prevent duplicate registrations.
- Reservation: PK = ReservationID (INT IDENTITY), FK to GuestID and RoomID. CHECK constraint on reservation_status ensures only valid status values are stored.
- Rooms: PK = RoomID (VARCHAR), FK to RoomTypeID and BuildingID. CHECK constraint on AvailabilityStatus ('available', 'occupied', 'maintenance').
- Invoice: PK = InvoiceID, FK to ReservationID. Computed fields (TotalRoomCharge, TotalServiceCharge, NetTotalAmount) are stored explicitly for reporting performance.

- Payments: PK = PaymentID, FK to InvoiceID and EmployeeID. CHECK constraint limits PaymentMethod to ['Cash','Card','Online','Wallet'].

3.2.2 Support Tables

- RoomsType: Master reference for room categories with NightlyPrice.
- Buildings: Two buildings with name, address, and floors.
- Services: Active/inactive flag (IsActive) allows decommissioning services without deleting historical usage.
- LoyaltyProgram: MembershipLevel CHECK constraint ['Bronze','Silver','Gold']. Auto-updated as TotalPoints accumulate.
- Employee: JobRole CHECK constraint ['Receptionist','Billing Manager','Admin','Housekeeping'].
- HousekeepingLogs: StartTime and EndTime stored as TIME for duration calculations.
- GuestsReview: Rating columns constrained BETWEEN 1 AND 5 to enforce valid ratings.

3.3 CRUD Operations

The database supports all four fundamental data operations:

3.3.1 INSERT (Create)

New guest records, reservations, and payments are inserted with full validation. For example, when a new reservation is confirmed, rows are inserted into Reservation, and a corresponding Invoice record is created in the same transaction to maintain referential integrity.

```
INSERT INTO Guests (GuestID, FullName, Phone, Email, City, NationalID\_Passport)
VALUES ('G31', 'Ahmed Kamal', '010-123-4567', 'ahmed@email.com', 'Cairo', 'NID-31001');
INSERT INTO Reservation (GuestID, RoomID, check\_in\_date, check\_out\_date, nights,
Adults\_Count, reservation\_status, Booking\_Source, booking\_date)
```

```
VALUES ('G31', 'R5', '2026-04-01', '2026-04-05', 4, 2, 'confirmed', 'Direct', GETDATE());
```

3.3.2 SELECT (Read)

All business queries use SELECT statements with JOINS, GROUP BY, and window functions.

Views were created for frequently needed queries [detailed in Section 4].

3.3.3 UPDATE (Modify)

Status updates are the most critical UPDATE operations — changing a reservation from 'confirmed' to 'checked_in', or marking an invoice as paid:

UPDATE Reservation

SET reservation_status = 'checked_in',

CheckedInByCustomerID = 'E2'

WHERE ReservationID = 49;

UPDATE Invoice

SET PaidAmount = PaidAmount + 2000.00

WHERE ReservationID = 49;

3.3.4 DELETE (Remove)

Deletes are used carefully. In production, we prefer soft deletes [status flags] over hard deletes to preserve audit trails. However, test data cleanup uses:

-- Soft delete: deactivate a service instead of removing it

UPDATE Services SET IsActive = 0 WHERE ServiceID = 'SV10';

-- Hard delete: remove a test reservation [cascade handled by FK constraints]

DELETE FROM Reservation WHERE ReservationID = 99 AND reservation_status = 'cancelled';

3.4 Bulk Insert

To populate the database with realistic test data (30 guests, 30 rooms, 28 employees, 75+ reservations), we used bulk INSERT scripts with multiple VALUES rows in a single statement.

This approach is far more efficient than individual INSERT calls for large datasets:

```
INSERT INTO Guests (GuestID, FullName, Phone, Email, City, NationalID\_Passport)
```

```
VALUES
```

```
['G1', 'John Doe', '011-111-0001', 'john@mail.com', 'Cairo', 'NID-10001'],
```

```
['G2', 'Sara Gad', '011-111-0002', 'sara@mail.com', 'Suez', 'NID-10002'],
```

```
['G3', 'Omar Nour', '011-111-0003', 'omar@mail.com', 'Giza', 'NID-10003'],
```

```
-- ... 27 more rows in a single INSERT batch
```

```
['G30', 'Jana I', '011-111-0030', 'jana@mail.com', 'Alex', 'NID-10030'];
```

This bulk approach ensures consistent, reproducible test data while minimizing database round-trips.

4. SQL Business Analysis — 10 Key Questions

After building the database foundation, we moved to answering critical business questions using SQL. Each question was addressed either through a reusable VIEW (for frequently needed data) or a direct analytical query (for one-time reports). Views encapsulate complex JOIN logic, making the data accessible to Power BI and application layers without exposing raw table complexity.

Q1: Guest Master View — Who Are Our Guests?

Business Context

The Guest Master View answers a fundamental question for hotel management: Who are our guests, and how valuable are they? This view combines the guest profile with their stay history and total lifetime spending — the key metrics for VIP identification, marketing segmentation, and loyalty program management.

Technical Logic

The view uses a LEFT JOIN between Guests and Reservations [to include guests who have never checked out], counts only completed stays [checked_out status], and sums payments with Completed status to calculate true lifetime spend. Results are ordered by LifetimeSpend descending to surface the most valuable guests first.

```
CREATE VIEW vw\_GuestMaster AS
SELECT
  g.GuestID, g.FullName, g.NationalID\_Passport,
  g.Phone, g.Email, g.City,
  -- Count of completed stays only
  COUNT(DISTINCT CASE WHEN r.reservation\_status = 'checked\_out'
    THEN r.ReservationID END) AS TotalStays,
  -- Sum of all actual payments made by the guest
  ISNULL(SUM(CASE WHEN p.PaymentStatus = 'Completed'
    THEN p.AmountPaid END), 0) AS LifetimeSpend
FROM Guests g
LEFT JOIN Reservation r ON g.GuestID = r.GuestID
LEFT JOIN Invoice i ON r.ReservationID = i.ReservationID
LEFT JOIN Payments p ON i.InvoiceID = p.InvoiceID
GROUP BY g.GuestID, g.FullName, g.NationalID\_Passport,
  g.Phone, g.Email, g.City;
```

Output & Insights

The query returned 30 guests ordered by spending. Top spenders: Samy A and Hanan S [Cairo/Alex] both at 5,100 EGP. Ola A and Salma I follow closely. Notably, G28 [Ola A] shows 0 TotalStays despite 5,000 EGP spend — indicating payments were made but the reservation status was not updated to checked_out, flagging a data quality issue.

Questions s...Admin (54))* X LAST Q.sql - not connected

```
-- =====  
-- Q1: Guest Master View  
-- =====  
-- View: combines guest profile + stay history + total spend  
-- LifetimeSpend = sum of all actual payments made by the guest  
-- TotalStays    = only fully checked_out reservations  
  
CREATE VIEW vw_GuestMaster AS  
SELECT  
    g.GuestID,  
    g.FullName,  
    g.NationalID_Passport,  
    g.Phone,  
    g.Email,  
    g.City,  
  
    -- Count of completed stays only  
    COUNT(DISTINCT CASE  
        WHEN r.reservation_status = 'checked_out'  
        THEN r.ReservationID  
    END) AS TotalStays,
```

110 % 10 0 ↑ ↓ Ln: 508, Ch: 5 MIXED CRLF UTF-8

Results Messages

	GuestID	FullName	City	TotalStays	LifetimeSpend
1	G13	Samy A	Cairo	1	5100.00
2	G22	Hanan S	Alex	1	5100.00
3	G28	Ola A	Suez	0	5000.00
4	G20	Salma I	Suez	1	4800.00
5	G5	Mona Aly	Cairo	1	4750.00
6	G26	Tala S	Alex	1	4700.00
7	G24	Ghada A	Suez	1	4700.00
8	G15	Yara S	Giza	1	4700.00
9	G7	Tamer H	Giza	1	4700.00
10	G30	Jana I	Alex	1	4200.00
11	G8	Noha G	Suez	1	3100.00
12	G10	Maya Z	Alex	1	3000.00
13	G25	Sherif	Cairo	0	2800.00
14	G4	Sara Gad	Suez	1	2750.00
15	G1	John Doe	Cairo	1	2600.00
16	G12	Huda S	Suez	0	2500.00
17	G14	Amira Z	Alex	1	2400.00
18	G27	Tamer G	Giza	1	2300.00
19	G19	Ramy S	Giza	1	2300.00

Q2: Room Availability View — What Rooms Are Free?

Business Context

Receptionists and online booking systems need real-time room availability data. This view provides a snapshot of all rooms with their type, pricing, current availability status, and any overlapping reservations — enabling precise filtering for any requested date range.

Technical Logic

The view joins Rooms with RoomsType for pricing information, and LEFT JOINs with Reservation to attach any active or future reservation dates. The LEFT JOIN is crucial — it ensures rooms without reservations still appear in the result. Filtering happens in the calling query, not the view itself, preserving flexibility.

```
CREATE VIEW vw\_RoomAvailability AS
SELECT
  ro.RoomID, ro.RoomNumber,
  rt.TypeName AS RoomType,
  rt.NightlyPrice AS NightlyRate,
  ro.AvailabilityStatus,
  ro.BuildingID,
  -- Attach reservation dates for external filtering
  r.ReservationID,
  r.check\_in\_date,
  r.check\_out\_date,
  r.reservation\_status
FROM Rooms ro
JOIN RoomsType rt ON ro.RoomTypeID = rt.RoomTypeID
LEFT JOIN Reservation r
ON ro.RoomID = r.RoomID
AND r.reservation\_status IN ('confirmed','checked\_in');
```

-- Usage: Find available rooms for a given date range

```
SELECT DISTINCT RoomID, RoomNumber, RoomType, NightlyRate, BuildingID
```

```
FROM vw\_RoomAvailability
```

```
WHERE ReservationID IS NULL
```

```
OR check\_out\_date <= '2026-04-01'
```

```
OR check\_in\_date >= '2026-04-05';
```

Output & Insights

The view shows 30 rooms across Building B1 and B2. The system contains 12 Double rooms [900 EGP/night], 12 Single rooms [500 EGP/night], and 6 Suite rooms [1,500 EGP/night]. This distribution suggests the hotel targets primarily budget-to-mid-range guests, with suites as premium offerings.

Questions.s...\Admin (54))*LAST Q.sql - not connected

567568569570571572573574575576577578579580581582583584585586587588

-- =====
-- Q2: Room Availability View (by date range)
-- =====
-- Shows every room with its type and nightly rate.
-- AvailabilityStatus in the view is the physical status from Rooms.
-- To check availability for a date range we filter in the query below.

CREATE VIEW vw_RoomAvailability AS
SELECT
 ro.RoomID,
 ro.RoomNumber,
 rt.TypeName AS RoomType,
 rt.NightlyPrice AS NightlyRate,
 ro.AvailabilityStatus,
 ro.BuildingID,

 -- Attach any active/future reservation dates so we can filter later
 r.ReservationID,
 r.check_in_date,
 r.check_out_date,
 r.reservation_status

110 %100012Ln: 585, Ch: 21MIXEDCRLFUTF-8

ResultsMessages

	RoomID	RoomNumber	RoomType	NightlyRate	BuildingID
1	R3	103	Double	900.00	B1
2	R4	104	Double	900.00	B1
3	R8	108	Double	900.00	B1
4	R9	109	Double	900.00	B1
5	R13	203	Double	900.00	B2
6	R14	204	Double	900.00	B2
7	R18	208	Double	900.00	B2
8	R19	209	Double	900.00	B2
9	R23	303	Double	900.00	B1
10	R24	304	Double	900.00	B1
11	R28	403	Double	900.00	B2
12	R29	404	Double	900.00	B2
13	R1	101	Single	500.00	B1
14	R2	102	Single	500.00	B1
15	R6	106	Single	500.00	B1
16	R7	107	Single	500.00	B1
17	R11	201	Single	500.00	B2
18	R12	202	Single	500.00	B2
19	R16	206	Single	500.00	B2

Q3: Daily Occupancy Rate — How Full Are We Each Day?

Business Context

Made with Napkin

Occupancy rate is the single most important KPI in hospitality. Hotels aim for 70–80% occupancy for profitability. This query calculates occupancy for every day in a specified date range, enabling trend analysis, identification of low-demand periods, and evidence-based pricing decisions.

Technical Logic

This is the most technically sophisticated query in the project. It uses a recursive CTE to generate a calendar of all dates in the range (avoiding dependency on a calendar table). A second CTE queries the total room inventory (excluding maintenance). The main query counts rooms with active reservations (checked_in or confirmed) covering each calendar date, then divides by total inventory.

```
DECLARE @RangeStart DATE = '2026-01-01';
DECLARE @RangeEnd DATE = '2026-03-17';
WITH Calendar AS [
SELECT @RangeStart AS CalDate
UNION ALL
SELECT DATEADD(day, 1, CalDate) FROM Calendar
WHERE CalDate < @RangeEnd
],
RoomInventory AS [
SELECT COUNT(*) AS TotalRooms
FROM Rooms WHERE AvailabilityStatus <> 'maintenance'
]
SELECT
c.CalDate AS Date,
ri.TotalRooms,
```

```
COUNT(r.ReservationID) AS OccupiedRooms,  
CAST(  
COUNT(r.ReservationID) \* 100.0 / NULLIF(ri.TotalRooms, 0)  
AS DECIMAL(5,2)) AS [OccupancyRate%]  
FROM Calendar c  
CROSS JOIN RoomInventory ri  
LEFT JOIN Reservation r  
ON c.CalDate >= r.check\_in\_date  
AND c.CalDate < r.check\_out\_date  
AND r.reservation\_status IN ('checked\_in','confirmed')  
GROUP BY c.CalDate, ri.TotalRooms  
ORDER BY c.CalDate  
OPTION (MAXRECURSION 400);
```

Output & Insights

Results show 28 total rooms with a consistent 1 occupied room (3.57% occupancy) throughout January–mid-March 2026. This extremely low occupancy is expected for a newly populated test database with limited reservations. In a real production environment, this query would reveal seasonal peaks and troughs, enabling revenue management strategies like dynamic pricing.

Questions .s... \Admin (54))* X LAST Q.sql - not connected

```

622
623 -- =====
624 -- Q3: Daily Occupancy Rate
625 -- =====
626 -- Generates one row per calendar date in the range.
627 -- OccupiedRooms = rooms that have an active (checked_in / confirmed)
628 -- reservation covering that specific date.
629
630 DECLARE @RangeStart DATE = '2026-01-01';
631 DECLARE @RangeEnd   DATE = '2026-03-17';
632
633 -- Build a calendar CTE for the date range
634 WITH Calendar AS (
635     SELECT @RangeStart AS CalDate
636     UNION ALL
637     SELECT DATEADD(day, 1, CalDate)
638     FROM   Calendar
639     WHERE  CalDate < @RangeEnd
640 ),
641 -- Total active rooms (not under maintenance)
642 RoomInventory AS (
643     SELECT COUNT(*) AS TotalRooms

```

110 % 10 0 ↑ ↓ Ln: 634, Ch: 19 MIXED CRLF UTF-8

Results Messages

	Date	TotalRooms	OccupiedRooms	OccupancyRate%
1	2026-01-01	28	1	3.57
2	2026-01-02	28	1	3.57
3	2026-01-03	28	1	3.57
4	2026-01-04	28	1	3.57
5	2026-01-05	28	1	3.57
6	2026-01-06	28	1	3.57
7	2026-01-07	28	1	3.57
8	2026-01-08	28	1	3.57
9	2026-01-09	28	1	3.57
10	2026-01-10	28	1	3.57
11	2026-01-11	28	1	3.57
12	2026-01-12	28	1	3.57
13	2026-01-13	28	1	3.57
14	2026-01-14	28	1	3.57
15	2026-01-15	28	1	3.57
16	2026-01-16	28	1	3.57
17	2026-01-17	28	1	3.57
18	2026-01-18	28	1	3.57
19	2026-01-19	28	1	3.57

Q4: Reservation Details View — Current Checked-In Guests

Business Context

The front desk needs a complete view of all current reservations — who is staying, in which room, for how long, and what they are being charged. This view serves as the operational backbone for check-in desks, housekeeping coordination, and billing verification.

Technical Logic

The view joins five tables: Reservation, Guests, Rooms, RoomsType, and Invoice. A CASE statement converts internal status codes ['confirmed', 'checked_in', etc.] into human-readable labels. The NightlyRateAtBooking field is pulled from the Invoice rather than the current RoomsType price, preserving historical accuracy if prices change.

```
CREATE VIEW vw\_ReservationDetails AS
SELECT
  r.ReservationID,
  g.FullName AS GuestName,
  ro.RoomNumber,
  rt.TypeName AS RoomType,
  r.check\_in\_date AS CheckInDate,
  r.check\_out\_date AS CheckOutDate,
  r.nights AS Nights,
  CASE r.reservation\_status
    WHEN 'confirmed' THEN 'Booked'
    WHEN 'checked\_in' THEN 'Checked-in'
    WHEN 'checked\_out' THEN 'Checked-out'
    WHEN 'cancelled' THEN 'Cancelled'
    WHEN 'no\_show' THEN 'No Show'
  END AS ReservationStatus,
  i.TotalRoomCharge,
```

```
r.Booking\_Source,  
rt.NightlyPrice AS NightlyRateAtBooking  
FROM Reservation r  
JOIN Guests g ON r.GuestID = g.GuestID  
JOIN Rooms ro ON r.RoomID = ro.RoomID  
JOIN RoomsType rt ON ro.RoomTypeID = rt.RoomTypeID  
LEFT JOIN Invoice i ON r.ReservationID = i.ReservationID;
```

Output & Insights

The query shows 6 currently checked-in guests (status = 'Checked-in'): Ali Omar in room 104 (Double, 7,200 EGP for 8 nights), Ziad S in 201 (Single, 2,000 EGP, 4 nights), Kareem A in 209 (Double), and three more guests. The NULL BookingSource values reveal a data entry issue — booking source was not captured for these reservations.

Questions s...\Admin (54))* LAST Q.sql - not connected

675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696

-- =====
-- Q4: Reservation Details View
-- =====

CREATE VIEW vw_ReservationDetails AS
SELECT
 r.ReservationID,
 g.FullName AS GuestName,
 ro.RoomNumber,
 rt.TypeName AS RoomType,
 r.check_in_date AS CheckInDate,
 r.check_out_date AS CheckOutDate,
 r.nights AS Nights,

 -- Human-readable status label
 CASE r.reservation_status
 WHEN 'confirmed' THEN 'Booked'
 WHEN 'checked_in' THEN 'Checked-in'
 WHEN 'checked_out' THEN 'Checked-out'
 WHEN 'cancelled' THEN 'Cancelled'
 WHEN 'no_show' THEN 'No Show'

110 % 10 0 Ln: 676, Ch: 1 (2 chars, 1 lines) MIXED CRLF UTF-8

Results Messages

	ReservationID	GuestName	RoomNumber	RoomType	CheckInDate	CheckOutDate	Nights	ReservationStatus	TotalRoomCharge	BookingSource	NightlyRateAtBooking
1	49	Ali Omar	104	Double	2026-02-25	2026-03-05	8	Checked-in	7200.00	NULL	900.00
2	52	Ziad S	201	Single	2026-02-28	2026-03-04	4	Checked-in	2000.00	NULL	500.00
3	55	Karem A	209	Double	2026-02-28	2026-03-05	5	Checked-in	4500.00	NULL	900.00
4	58	Huda S	402	Single	2026-03-01	2026-03-05	4	Checked-in	2000.00	NULL	500.00
5	65	Wael H	302	Single	2026-03-01	2026-03-05	4	Checked-in	2000.00	NULL	500.00
6	72	Ola A	108	Double	2026-03-01	2026-03-05	4	Checked-in	3600.00	NULL	900.00

Q5: Cancellation Analysis — Why Are Guests Cancelling?

Business Context

Cancellations directly reduce revenue and create operational uncertainty. Understanding cancellation patterns by month and reason enables targeted interventions: flexible cancellation policies for high-cancellation periods, deposits for high-risk booking channels, and process improvements to address specific cancellation reasons.

Technical Logic

The query uses two analytical approaches: [1] Monthly summary using CASE-based conditional aggregation to count cancellations vs. total reservations and calculate the cancellation rate percentage, [2] Top cancellation reasons using TOP with GROUP BY and ORDER BY. The NULLIF prevents division-by-zero errors.

-- Monthly cancellation performance

```
SELECT
FORMAT(r.booking\_date, 'yyyy-MM') AS YearMonth,
COUNT(\*) AS TotalReservations,
SUM(CASE WHEN r.reservation\_status = 'cancelled'
THEN 1 ELSE 0 END) AS CancelledReservations,
CAST(SUM(CASE WHEN r.reservation\_status = 'cancelled'
THEN 1.0 ELSE 0 END)
\* 100.0 / NULLIF(COUNT(\*), 0)
AS DECIMAL[5,2]) AS [CancellationRate%]
FROM Reservation r
GROUP BY FORMAT(r.booking\_date, 'yyyy-MM')
ORDER BY YearMonth;
```

```
-- Top 5 cancellation reasons
SELECT TOP 5
ISNULL(CancellationReason,'No reason provided') AS CancellationReason,
COUNT(\*) AS CancellationCount
FROM Reservation
WHERE reservation\_status = 'cancelled'
GROUP BY CancellationReason
ORDER BY CancellationCount DESC;
```

Output & Insights

Monthly data shows: Oct-2025 [1 reservation, 0% cancellation], Nov-2025 [4 reservations, 0%], Dec-2025 [4 reservations, 0%], Jan-2026 [9 reservations, 11.11% cancellation rate — 1 cancelled], Feb-2026 [10 reservations, 0%]. Only 1 total cancellation exists in the dataset, with 'No reason provided' as the reason. This points to a system gap: the cancellation reason field should be mandatory to generate useful analytics.

Parse (Ctrl+F5)

Admin (54))*

LAST Q.sql - not connected

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

-- =====

-- Q5: Cancellation Analysis

-- =====

-- Monthly cancellation performance

SELECT

FORMAT(r.booking_date, 'yyyy-MM') AS YearMonth,

COUNT(*) AS TotalReservations,

SUM(CASE WHEN r.reservation_status = 'cancelled' THEN 1 ELSE 0 END)

AS CancelledReservations,

CAST(

SUM(CASE WHEN r.reservation_status = 'cancelled' THEN 1.0 ELSE 0 END)

* 100.0 / NULLIF(COUNT(*), 0)

AS DECIMAL(5,2)) AS [CancellationRate%]

FROM Reservations r

GROUP BY FORMAT(r.booking_date, 'yyyy-MM')

ORDER BY YearMonth;

GO

-- Top 5 cancellation reasons

SELECT TOP 5

110 %

10

0

↑

↓

Ln: 736, Ch: 3

MIXED

CRLF

UTF-8

Results

Messages

	YearMonth	TotalReservations	CancelledReservations	CancellationRate%
1	2025-10	1	0	0.00
2	2025-11	4	0	0.00
3	2025-12	4	0	0.00
4	2026-01	9	1	11.11
5	2026-02	10	0	0.00

	CancellationReason	CancellationCount
1	No reason provided	1

Q6: Services Revenue View — Which Services Drive Income?

Business Context

Ancillary services [spa, laundry, dining] can contribute 20–30% of total hotel revenue.

Understanding which services generate the most revenue per month allows management to invest in high-performing services, promote underutilized offerings, and optimize staffing in service departments.

Technical Logic

The view aggregates ServiceUsage by service name and year-month, computing both usage count and total revenue. The analytical query then pulls the top 5 services by revenue over the last 3 months using a date filter (DATEADD), sorting by total revenue descending.

```
CREATE VIEW vw\_ServiceRevenue AS
SELECT
s.ServiceName,
FORMAT[su.UsageDate, 'yyyy-MM'] AS YearMonth,
COUNT[su.UsageID] AS TotalUsageCount,
SUM[su.TotalCharge] AS TotalServiceRevenue
FROM ServiceUsages su
JOIN Services s ON su.ServiceID = s.ServiceID
GROUP BY s.ServiceName, FORMAT[su.UsageDate, 'yyyy-MM'];
-- Top 5 services by revenue in the last 3 months
SELECT TOP 5
```

```
ServiceName,  
SUM[TotalUsageCount] AS UsageCount,  
SUM[TotalServiceRevenue] AS Revenue  
FROM vw\_ServiceRevenue  
WHERE YearMonth >= FORMAT[DATEADD(month,-3,GETDATE())],'yyyy-MM')  
ORDER BY Revenue DESC;
```

Output & Insights

Top 3 services in the last 3 months: Spa leads with 10 usages and 5,500 EGP revenue, followed by Laundry (10 usages, 2,300 EGP), and Dinner (5 usages, 2,100 EGP). The high Spa revenue despite equal usage count to Laundry confirms Spa has a higher per-use charge. This suggests the hotel should invest in Spa capacity and possibly develop Spa packages to increase revenue further.

Field	Description
Rows (Records)	~75+ reservation records covering the full operational period [Oct 2025 – Mar 2026]
Columns (Features)	20+ features including booking dates, stay duration, guest demographics, room type, pricing, status, and payment data
Data Types	Mix of categorical (room type, booking source, status), numerical (nights, charges, amount), and datetime (check-in/out dates)

Field	Description
reservation_status	Categorical. Target variable. Values: confirmed, checked_in, checked_out, cancelled, no_show
nights	Numeric [integer]. Duration of stay. Range: 1–14 nights. Key predictor of revenue
Adults_Count	Numeric [integer]. Number of adult guests. Affects service usage patterns
Children_Count	Numeric [integer]. Number of child guests. Often 0 [many solo/couple bookings]
Total_Price	Numeric [float]. Total room price = nights × nightly rate. High correlation with nights and room type
Discounts	Numeric [float]. Discount applied to the booking. Can be 0 or a percentage reduction
Booking_Source	Categorical. How the booking was made: Direct, OTA, Phone, Walk-in
check_in_date / check_out_date	Datetime. Used to derive day-of-week, month, and season features
RoomType	Categorical [derived]. Single, Double, Suite. Key segmentation variable
NightlyRate	Numeric. Room price per night. Determined by RoomType
TotalRoomCharge	Numeric. Invoice-level charge. Equals nights × NightlyRate [before discounts]
AmountPaid	Numeric. Actual payment received. Difference from TotalRoomCharge = outstanding
City	Categorical. Guest's home city [Cairo, Alexandria, Suez, Giza]. For geographic segmentation
Booking_Date	Datetime. When the booking was made. Lead time = check_in_date - booking_date

Field	Description
nights (stay duration)	Mean $\approx$ 4.8 nights, Median $\approx$ 4 nights. Slight right skew — some long stays pull the mean up. Most guests stay 3–5 nights.
Total_Price	Mean and median close, slight right skew from Suite bookings. Majority of bookings fall in the 1,500–4,500 EGP range.
Adults_Count	Mode = 2 [couples most common]. Distribution: 1 adult [30%], 2 adults [55%], 3+ adults [15%].

Field	Description
CSV File Import	Entry point. Loads the hotel booking dataset (CSV export from SQL Server). 75+ records, 20+ features.
Edit Domain	Renames and reclassifies columns. Sets reservation_status as the target (class) variable and marks non-predictive IDs for exclusion.
Preprocess	Handles remaining missing values using the median/mode strategy, applies feature normalization (standardization for numeric features), and encodes categorical.
Select Columns	Selects the most relevant features identified during EDA: nights, roomType, Booking_Source, Adults_Count, Discounts, NightlyRate, City, check_in month, lead_time.
Data Sampler	Splits data into 80% training / 20% testing using stratified sampling to preserve class proportions.
Violin Plot / Box Plot / Distributions	EDA visualization widgets. Used to explore feature distributions visually before modeling.
Correlations + Data Table (2)	Computes and displays correlation matrix. Data Table (2) shows preprocessed data for inspection.
Outliers widget	Identifies statistical outliers using IQR/z-score method. Connected to 'Data after removing Outliers' for a cleaned version.
k-Means	Unsupervised clustering to identify natural guest segments based on booking behavior.
Scatter Plot	Visualizes cluster assignments in 2D feature space to interpret segment characteristics.

Field	Description
Random Forest	An ensemble of 100 decision trees. Each tree is trained on a random subset of features and data. Final prediction is majority vote. Best performer in our workflow (reported accuracy: 81%).
Gradient Boosting	Sequential ensemble where each new tree corrects the errors of the previous. Strong performer on tabular data with class imbalance.
SVM (Support Vector Machine)	Finds the optimal hyperplane separating classes. Effective in high-dimensional spaces. More sensitive to feature scaling — benefits from our normalization step.
AdaBoost	Adaptive Boosting. Assigns higher weights to misclassified samples in each iteration. Good at reducing bias.
Decision Tree	Single tree model. Interpretable but prone to overfitting. Serves as the baseline model.
Logistic Regression	Linear model for classification. Fast and interpretable. Baseline for linear separability assumption.
Neural Network	Multi-layer perceptron. Captures non-linear interactions. Requires more data to outperform ensemble methods.
kNN (k-Nearest Neighbors)	Classifies based on the k most similar training examples. Simple but sensitive to irrelevant features.

Field	Description
Random Forest	Best Overall — 81% accuracy. Low false-negative rate for ‘cancelled’ class. Robust to outliers and class imbalance.
Gradient Boosting	Close second. Slightly better recall for minority classes (cancelled, no_show) but slower training.
SVM	Good precision but lower recall for imbalanced classes. Competitive overall.
Logistic Regression	Baseline linear model. Decent for majority class but struggles with minority class prediction.
Decision Tree	Overfits on training data (100% training accuracy) but drops to ~70% on test set. Prune needed .
Neural Network	Competitive but requires more data to outperform Random Forest in this dataset size.
kNN	Sensitive to the high dimensionality of encoded categoricals. Lowest performer.

Field	Description
Occupancy Rate	Currently ~3.57% (test database). Target: 70–80% for profitability. Formula: Occupied Rooms / Total Rooms × 100.
Average Daily Rate (ADR)	Average revenue per occupied room per night. Formula: Total Room Revenue / Occupied Room Nights.
RevPAR (Revenue per Available Room)	ADR × Occupancy Rate. The gold standard hotel KPI combining both rate and occupancy.
Average Length of Stay (ALOS)	Mean stay duration in nights. Currently ~4.8 nights. Longer stays = higher revenue per booking.
Cancellation Rate	Currently ~3.5% (1/28 reservations). Industry average: 20–40% for OTA bookings. Our low rate reflects the small dataset.
Total Ancillary Revenue	Service revenue (Spa + Laundry + Food) as % of total revenue. Target: 20–30%.
Outstanding Balance Ratio	Unpaid invoices as % of total invoiced amount. Target: < 5%.
Guest Satisfaction Score	Average of StaffService + Cleanliness + Overall ratings. Target: > 4.0/5.0.

9. Final Insights & Business Recommendations

9.1 Business Insights

- Revenue concentration risk: The top 3 guests (Samy A, Hanan S, Ola A) account for a disproportionate share of lifetime spend. While valuable, this concentration means losing a handful of VIP guests would significantly impact revenue.
- Spa is an underexploited revenue driver: With only 10 usages and 5,500 EGP in the dataset, Spa is already the top service by revenue per use. Increasing Spa capacity and visibility could add significant ancillary revenue.
- January is the peak operational month: January 2026 showed the highest booking volume and total payment processing. Resource planning (staffing, supplies, maintenance windows) should account for this peak.
- Walk-in bookings carry higher operational risk: Cross-tabulation reveals Walk-in guests have higher no-show rates. This booking channel should require upfront payment or deposit.
- Building 82 has slightly more late housekeeping sessions: Rooms in Building 82 (R12, R13, R26, R27) show more late cleaning flags. This may indicate understaffing in 82 or a scheduling bottleneck.
- Direct booking channel is most profitable: Direct bookings have higher average spend and lower cancellation rates compared to OTA bookings, making investment in the hotel's direct booking platform more ROI-efficient than OTA partnerships.
- Random Forest model identifies nights and nightly rate as top predictors: Guests booking longer stays at higher rates are less likely to cancel — they have more invested in the trip. Short, cheap bookings need more monitoring.

9.2 Actionable Recommendations

Revenue Optimization

- Implement dynamic pricing for Double rooms during peak weekends (Friday–Saturday) to capture willingness-to-pay. Use the daily occupancy CTE to identify price increase opportunities.
- Create Spa + Room packages for Suite guests — they already spend more and are more likely to use premium services.
- Launch a direct booking incentive (5% discount or free room upgrade) to shift OTA bookings to direct channel, saving commission costs.

Occupancy Improvement

- Target Cairo and Alexandria corporate accounts for mid-week business travel (Mon–Thu), where current occupancy is lowest.
- Develop 2-night weekend packages for Suez guests (currently the second largest guest origin city) — proximity makes them ideal for short leisure breaks.
- Implement last-minute booking discounts (48 hours before arrival) for rooms that remain unoccupied — 10% discount is better than 0% occupancy.

Cancellation & No-Show Reduction

- Require a non-refundable deposit (20% of total) for Walk-in and OTA bookings with check-in within 48 hours — this segment has the highest no-show rate.
- Send automated pre-arrival reminders 3 days and 1 day before check-in to confirmed reservations.
- Make the CancellationReason field mandatory in the booking system — current data is insufficient for targeted intervention.

Data Quality Improvements

- Make Booking1_Source a required field in the reservation form — 15% NULL rate prevents channel analysis.
- Implement an automated reservation_status update trigger: when a payment is completed and dates have passed, automatically update status to checked_out.
- Create a post-stay review prompt (email or SMS sent 24 hours after checkout) to increase GuestsReview response rate.

Staff Optimization

- Assign specific reservations to specific receptionists and track completion — CheckedInByEmployeeID has too many NULLs currently.
- Investigate why Fady Aly, Maya Ihab, and Karem Gad show zero activity — reassign or investigate if roles need adjustment.
- Schedule housekeeping reinforcement for Building 82 to reduce late cleaning sessions, particularly after weekend checkouts.

10. Presentation Guide — 20 Slides

10.1 Suggested Slide Structure

Field	Description
Slide 1	Title Slide — Project name, team, tools (SQL Server, Python, Orange, Power BI)
Slide 2	Project Overview — Problem statement, objectives, data pipeline diagram (Database → EDA → ML → Dashboard)
Slide 3	ERD Overview — High-level entity diagram with 13 entities, brief explanation of core relationships
Slide 4	Entity Deep Dive — Reservation as central entity with all its connections (Guests, Rooms, Invoice, ServiceUsage)
Slide 5	Database Implementation — Table count, row counts after bulk insert, sample INSERT/UPDATE queries
Slide 6	Q1 & Q2 — Guest Master View output table + Room Availability screenshot
Slide 7	Q3 & Q4 — Daily Occupancy Rate CTE explanation + Reservation Details view results
Slide 8	Q5 & Q6 — Cancellation Analysis monthly table + Services Revenue top 5 bar
Slide 9	Q7 & Q8 — Invoice Aging view structure + Payment Method Breakdown monthly table
Slide 10	Q9 & Q10 — Housekeeping Performance table + Staff Performance view results
Slide 11	Transition Slide — Explain the move from SQL to Data Science with pipeline diagram
Slide 12	EDA: Dataset & Data Quality — Shape, missing values table, cleaning decisions
Slide 13	EDA: Distributions — Histograms (nights, Total_Price) and categorical pie charts (room type, booking source, status)
Slide 14	EDA: Relationships — Correlation heatmap + scatter plot (nights vs. Total_Price with room type)
Slide 15	EDA: Key Insights — 5 bullet points with supporting charts
Slide 16	Orange Workflow — Screenshot of .ows workflow with labels explaining each widget group
Slide 17	Model Comparison — Table of 8 models with accuracy, F1-score; highlight Random Forest as best
Slide 18	Power BI Dashboard — Screenshots of 2–3 dashboard pages with KPI highlights
Slide 19	Final Insights — Top 7 business insights from the full analysis (icons + concise text)
Slide 20	Recommendations & Next Steps — 3 categories (Revenue, Occupancy, Data Quality) with actionable bullet points

10.2 Live Demo Workflow (5 Minutes)

For a live demonstration during the presentation, follow this sequence:

1. SQL Server (2 min): Open SSMS, run Q3 (Daily Occupancy) to show the recursive CTE in action. Show the Results tab with the date-by-date output. Then open Q6 (Service Revenue) and highlight the top 3 services.
2. Jupyter Notebook (1 min): Show the EDA notebook with 2–3 key visualizations: the nights histogram, Total_Price box plot by room type, and the correlation heatmap.
3. Orange (1 min): Open Hotel_Project_updated.ows. Click on Test and Score to show the 81% accuracy. Click on Confusion Matrix to show which classes are predicted correctly.
4. Power BI (1 min): Open Hotel_Reservation.pbix. Show the Executive Summary page. Use a slicer to filter by Room Type = 'Suite' and demonstrate how all KPIs update dynamically.

10.3 Key Messages to Emphasize

- This is a full end-to-end data project — from raw database design to a live BI dashboard.
- Every tool in the pipeline was chosen deliberately: SQL for data integrity, Python for deep EDA, Orange for machine learning, Power BI for business consumption.
- The project is grounded in real business questions, not just technical exercises — every query and visualization answers a decision that a hotel manager actually faces.
- Random Forest with 81% accuracy can realistically be deployed to flag high-risk reservations for proactive intervention, reducing no-shows and cancellations.

11. Conclusion

This project demonstrated a complete data pipeline for a Hotel Reservation and Billing Management System — from conceptual database design through analytical SQL, exploratory data science, machine learning modeling, and business intelligence dashboarding.

The ERD captures 13 entities and their complex relationships, reflecting the real operational complexity of a hotel. The SQL implementation in SQL Server with proper constraints, views, and CTEs transforms raw data into actionable business intelligence. The 10 analytical queries answer the critical questions that hotel managers face daily: who are the most valuable guests, which rooms are available, what is driving cancellations, and who on the staff is performing best.

The Python EDA revealed important data quality issues (missing booking sources, uncaptured cancellation reasons) alongside genuine business patterns (Spa revenue leadership, January peak, Direct channel profitability). The Orange Data Mining workflow benchmarked 8 machine learning models, with Random Forest achieving 81% accuracy in predicting reservation outcomes — a model that, with further data collection, could be deployed to identify at-risk bookings before they become cancellations.

The Power BI dashboard synthesizes all these insights into an interactive management tool, with real-time KPI tracking for occupancy rate, revenue, payment methods, and housekeeping performance. Most importantly, this project is not just a technical exercise — every component was designed with a real business question in mind. The recommendations derived from the analysis (dynamic pricing, Spa packages, direct booking incentives, mandatory field enforcement) provide a concrete roadmap for improving hotel profitability and guest satisfaction.